

# CPE 486/586: Deep Learning for Engineering Applications

08 Diffusion Models

Spring 2026

**Rahul Bhadani**

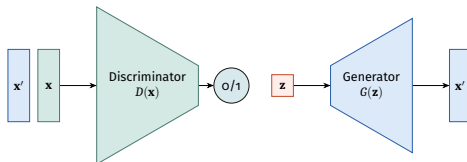
# Outline

1. Diffusion Models
2. Mathematical Framework for Diffusion Models
3. Latent Diffusion Models
4. ODE, Flow, and Dynamical Systems

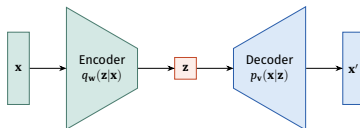
# Motivation

# Generative Models

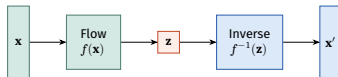
**GAN:** Adversarial training



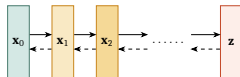
**VAE:** maximize variational lower bound



**Flow-based models:** Invertible transform of distributions



**Diffusion models:** Gradually add Gaussian noise and then reverse





# Why Diffusion Model and not VAE

- ⚡ VAE operates on a single-step compression - expansion idea.
- ⚡ High-frequency details like textures, sharp texts, etc are discarded in attempt to minimize the loss.

Diffusion Models attempt to solve this by breaking down each step into multiple tiny steps.

# Diffusion Model as the State of the Art for Generative Models



**Figures created using Midjourney v4 and GPT-4.**

# Diffusion Model are non-Adversarial

**Adversarial Models such as GAN are difficult to train.**

**100**

**Discriminator can be too good!!**

# Diffusion Models

# Inspiration behind Diffusion Model

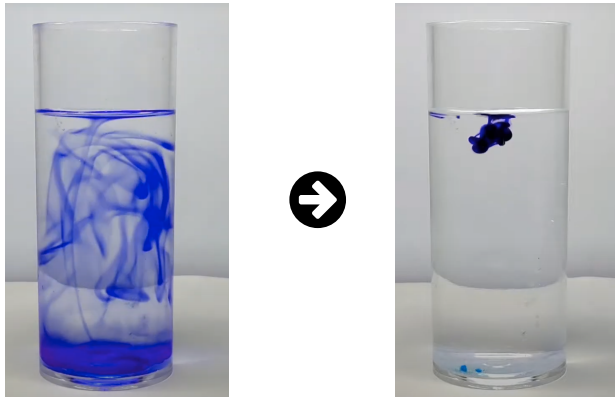
Inspired by non-equilibrium behavior in Thermodynamics



The drop of ink eventually diffuses into the water until it reaches equilibrium.

# Intuition behind Diffusion Model

**In the physical world, the reversal of the diffusion process is not possible. But that's what the diffusion model is trying to do.**



# Intuition behind Diffusion Model

**In the ink-water diffusion example, the drop of the ink at one spot includes some information.**

**As the diffusion process progresses, we lose information, and eventually, we only have noise.**

# Intuition behind Diffusion Model

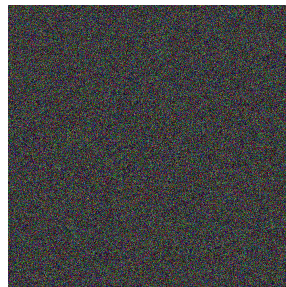
In the image domain:



Clear information



Some loss of information



noise

Just

**Working backward from just noise to a proper image is what a diffusion model tries to achieve.**

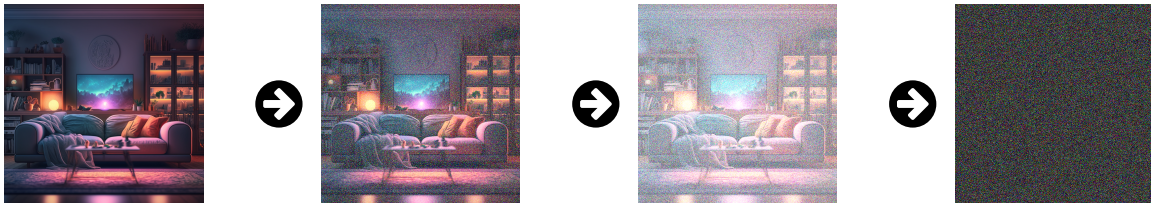


# The Working Principle of the Diffusion Model

**Add noise to the original image and then later learn how to reverse this noise process.**

# The Working Principle of the Diffusion Model

**Apply noise following a Markov chain/Markov Process.**

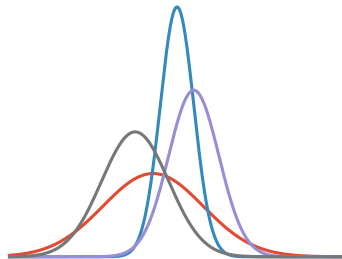


**At each time step, we add a little bit of noise to the image until the image only consists of the noise. We have 100-1000 long Markov chain.**

# Noise in the Diffusion Model

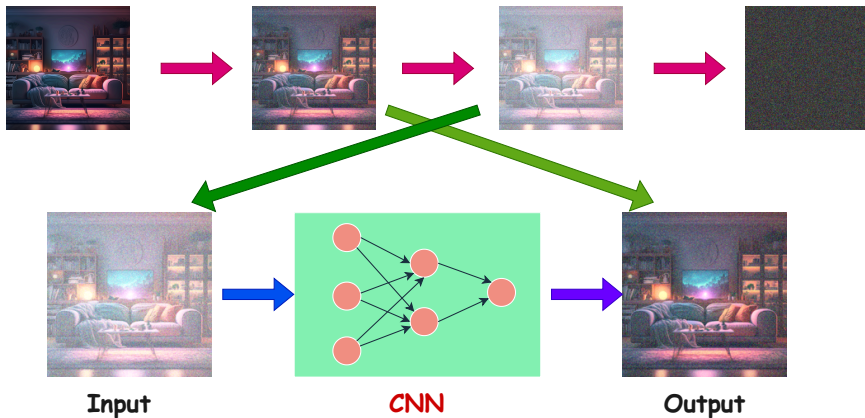
**Gaussian Noise:**  $\mathcal{N}(\mu, \sigma^2)$

$$\begin{array}{|c|} \hline 25 \\ \hline 67 \\ \hline \end{array} + \mathcal{N}(\mu, \sigma^2) = \begin{array}{|c|} \hline 26.63 \\ \hline 66.55 \\ \hline \end{array}$$



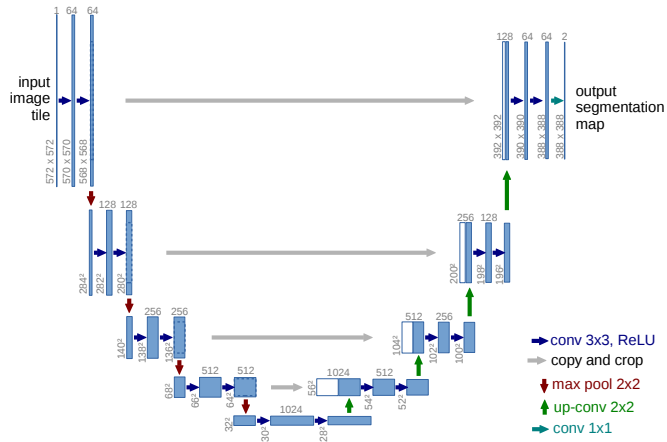
Repeat the process.

# Idea Behind Removing the Noise (Working Backward)



The type of CNN used in the original paper is called UNet. (Transformer can also be used). In general, we call them backbone models.

# UNet Architecture

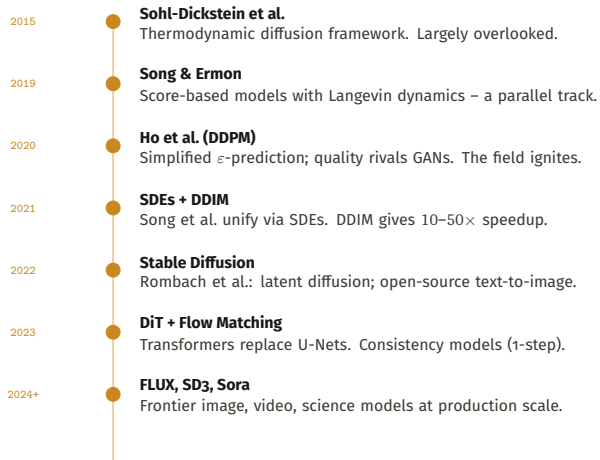


Source: <https://arxiv.org/pdf/1505.04597.pdf>

# UNet Architecture for Diffusion

- ⚡ U-net consists of a series of up/down sampling convolution blocks which are connected via residual connection, along with a middle block. This set up allow allows one to global location and context at the same time in the learning process. It works with very few training samples and provides better performance for segmentation tasks.
- ⚡ In the Diffusion model, UNet architecture is combined with time-step embedding called sinusoidal embedding. This is similar to the positional embedding seen in the transformer. This block combines sinusoidal embedding and convolution layers to learn the time step of the noisy images in the diffusion process.

# Historical Timeline



# Mathematical Framework for Diffusion Models: Denoising Diffusion Probabilistic Model (DDPM)



# Generated Samples from DDPM



Source: Denoising Diffusion Probabilistic Models

[https:](https://proceedings.neurips.cc/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf)

[//proceedings.neurips.cc/paper/2020/file/  
4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf](https://proceedings.neurips.cc/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf)

## Definition

A diffusion probabilistic model (a **diffusion model** for brevity) is a parameterized Markov chain trained using variational inference to produce samples matching the data after finite time.

Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." Advances in neural information processing systems 33 (2020): 6840-6851.

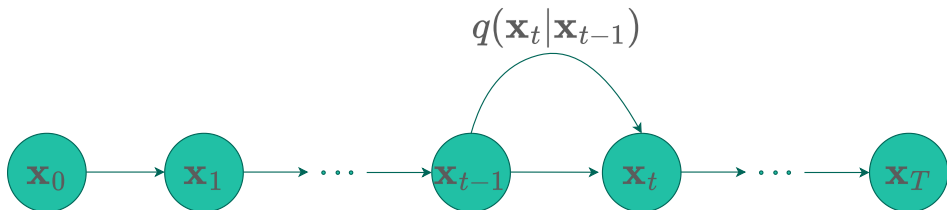
# Background Terminology

- ⚡  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ : the original data.
- ⚡  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ : latent dimensions of the same dimensionality as  $\mathbf{x}_0$ .
- ⚡ Joint distribution of the reverse process:  $p_\theta(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$  or  $p_\theta(\mathbf{x}_{0:T})$  in short.
- ⚡ Reverse process is defined as Markov chain with a learned Gaussian transition start at  $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T, \mathbf{0}, \mathbf{I})$ .
- ⚡ Approximate posterior  $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ , called forward process or diffusion process in the realm of diffusion model.
- ⚡ The approximate posterior is fixed to a Markov process (or Markov chain) that adds Gaussian noise little by little according to a variance schedule  $\beta_1, \beta_2, \dots, \beta_T$ .

# Formulating Diffusion Model: Forward Process

- ⚡ Consider an original image data sampled from the real distribution  $\mathbf{x}_0 \sim q(\mathbf{x})$ .
- ⚡ We add small Gaussian noise to the sample data in  $T$  steps, producing a sequence of noisy samples  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ .
- ⚡ The step sizes are controlled by the diffusion rate  $\{\beta_t \in (0, 1)\}$ , a hyperparameter.
- ⚡ The data sample  $\mathbf{x}_0$  slowly loses its distinguishable features as the step  $t$  becomes larger.

## Forward Process: Adding Noise



- ⚡ If the noise to be added is Gaussian, then we use the analytical distribution (also known as **transition kernel**) of the noisy data at time step  $t$  as  $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$ . **Note: this is a design choice.**
- ⚡ The conditional distribution of the noisy data for the subsequent Markov chain (i.e. joint distribution of  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ ) is written as

$$q(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T | \mathbf{x}_0) = q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=0}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$$

## Forward Process: Adding Noise

- ⚡ We sample  $\mathbf{x}_t$  at any arbitrary time step  $t$  in a closed form using reparameterization trick

$$\alpha_1 = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{i=1}^t \alpha_i.$$

This can help us write analytical form of  $q(\mathbf{x}_t|\mathbf{x}_0)$  for all  $t \in \{0, 1, \dots, T\}$  as

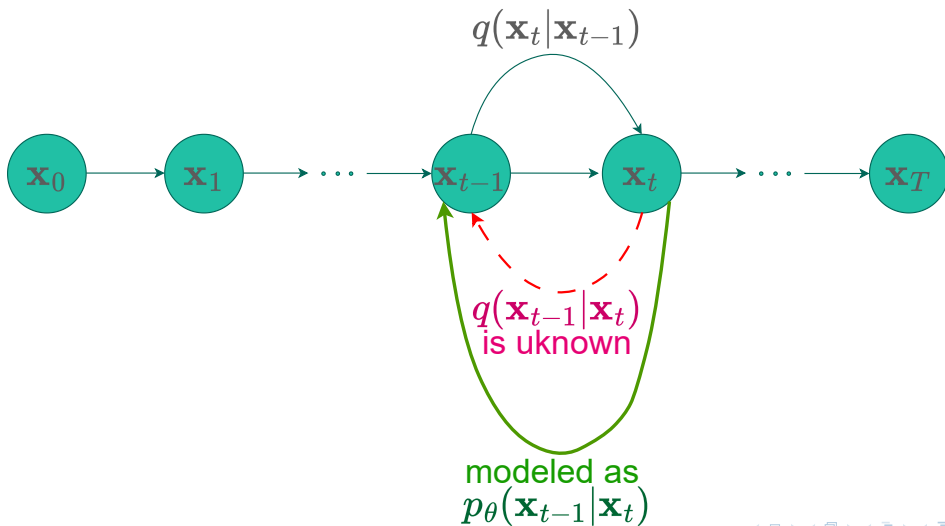
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}).$$

- ⚡ Given  $\mathbf{x}_0$ , we can sample  $\mathbf{x}_t$ , by supplying small Gaussian perturbation  $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$  and applying a transformation

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)}\varepsilon.$$

Intuitively speaking, this forward process slowly injects noise into data until all structures are lost.

# Reverse Diffusion Process: Removing Noise



## Reverse Diffusion Process: Removing Noise

- ⚡ We need to learn a model  $p_\theta$  to approximate these conditional probabilities  $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$  to run the reverse diffusion process.
- ⚡ We start with choosing the prior distribution  $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; 0, \mathbf{I})$  (because in forward process, we end up with  $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; 0, \mathbf{I})$ ).
- ⚡ The learnable transition kernel  $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$  can be obtained using a neural network model.
- ⚡ The learnable transition kernel can be mathematically written as a parametrized equation

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)).$$

$\boldsymbol{\mu}_\theta$  and  $\boldsymbol{\Sigma}_\theta$  are parameters to be learned using a neural network (UNet). An easier option is to set  $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$ .

- ⚡ With this reverse Markov chain in hand, we can generate a data sample  $\mathbf{x}_0$  by first sampling a noise vector  $\mathbf{x}_T \sim p(\mathbf{x}_T)$ , then iteratively sampling from the learnable transition kernel  $\mathbf{x}_{t-1} \sim p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$  until  $t = 1$ .

# Data Scaling

- ⚡ Image data are between 0 to 255 in integer. We scale them to  $[-1, 1]$ .
- ⚡ Since, image pixels are discrete, we want to obtain discrete log likelihood.
- ⚡ For that, we set the last term of the reverse process to an independent discrete decoder derived from the Gaussian  $\mathcal{N}(\mathbf{x}_0, \boldsymbol{\mu}_\theta(\mathbf{x}_1, 1), \sigma_1^2 \mathbf{I})$ :

$$p_\theta = \prod_{i=1}^D \int_{\delta_-(x_0^i)}^{\delta_+(x_0^i)} \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_\theta^i(\mathbf{x}_1, 1), \sigma_1^2) d\mathbf{x} \quad D \text{ is the data dimensionality}$$

and

$$\delta_+(x) = \begin{cases} \infty & \text{if } x = 1 \\ x + \frac{1}{255} & \text{if } x < 1 \end{cases}$$

$$\delta_-(x) = \begin{cases} -\infty & \text{if } x = -1 \\ x - \frac{1}{255} & \text{if } x > -1 \end{cases}$$



# What about the Loss Function?

- ⚡ The primary objective in training the UNet is the reverse Markov chain to match the actual time reversal of the forward Markov chain.
- ⚡ We need to adjust the parameter  $\theta$  in such a way so that joint probability distribution of the reverse Markov chain  $p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = p(\mathbf{x}_T) \prod_{i=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$  should closely approximate of the forward process  $q_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = q(\mathbf{x}_0) \prod_{i=1}^T q_{\theta}(\mathbf{x}_t|\mathbf{x}_{t-1})$ .
- ⚡ This is done by minimizing the distance between these two joint probability distributions using **Kullback-Leibler (KL) divergence**.

# KL Divergence as Loss Function

⚡ KL divergence for training the diffusion model is written as

$$\begin{aligned}\text{KL}(q, p) &= -\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)}[\log p_\theta(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)] + \text{constant} \\ &= \mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} \left[ -\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)}{q(\mathbf{x}_t | \mathbf{x}_{t-1})} \right] + \text{constant} \\ &\geq \mathbb{E}_{q(\mathbf{x}_0)}[-\log p_\theta(\mathbf{x}_0)] + \text{constant}\end{aligned}$$

The last inequality comes from Jensens's Inequality, i.e.

$$\mathbb{E}[g(X)] \geq g(\mathbb{E}[X])$$

$\mathbb{E}_{q(\mathbf{x}_0)}[-\log p_\theta(\mathbf{x}_0)]$  is variational upper bound (or negative ELBO as seen in VAE).

Two random variables  $A$  and  $B$  are related by  $P(A, B) = P(A)P(B|A)$ . Hence,  $q(\mathbf{x}_0, \mathbf{x}_{1:T}) = q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)$  which can be used as alternative expression above.

# Parametrizing the Loss Function

We can rewrite  $KL$  shown previously as

$$\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} \left[ \underbrace{KL(q(\mathbf{x}_T | \mathbf{x}_0) || p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{KL(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) || p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right]$$

This equation uses KL divergence to directly compare  $p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$  against forward process posteriors  $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$  which are tractable when conditioned on  $\mathbf{x}_0$ :

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

with  $\tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1-\beta_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t$  and  $\tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$ .

# Parametrizing the Loss Function

Both  $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$  and  $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$  are not Normal distribution, leading to a simpler KL divergence:

$$L_{t-1} = KL(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q \left[ \frac{1}{2\sigma_t^2} ||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2 \right] + C$$

We already have one reparameterization:  $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)}\epsilon$ .

Then,

$$\tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1 - \beta_t}} \left( \mathbf{x}_t - \frac{\beta}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

So the mean of denoising model can be learnt with a denoising network

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left( \mathbf{x}_t - \frac{\beta}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

# Simplified Loss Function

With the parameterization, we have

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[ \underbrace{\frac{\beta_t^2}{2\sigma_t^2(1 - \beta_t)(1 - \bar{\alpha}_t)}}_{\lambda_t} \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] + \mathcal{C}$$

The time dependent  $\lambda_t$  enables the training objective to be weighted for the maximum data likelihood training.

The original paper simplifies even further, and set  $\lambda_t = 1$ , and hence

$$L_{simple} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[ \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] + \mathcal{C}$$

# Training and Sample Generation Algorithm

## Algorithm 1 Training

---

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
6:    $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
7: until converged
```

---

## Algorithm 2 Sampling

---

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$   
        $+ \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

---

# Latent Diffusion Models

# Latent Diffusion Model

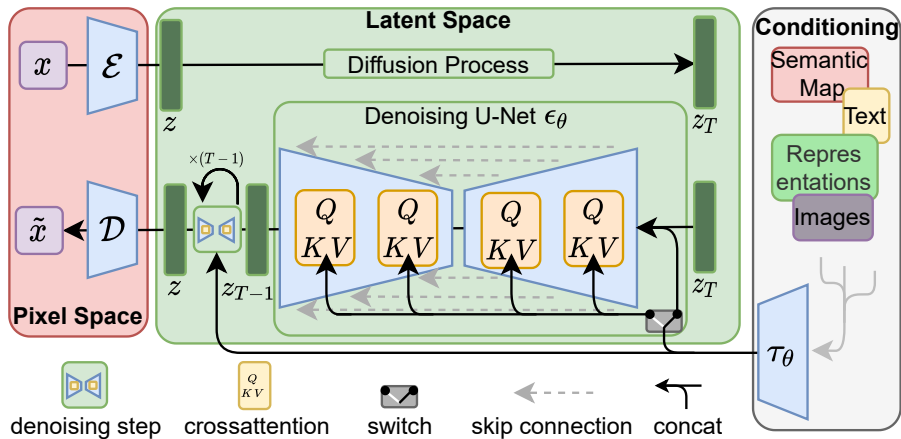
**The diffusion model runs in a latent space instead of a pixel space, making training costs lower and inference faster.**

Latent: hidden or compression space (like PCA)!

**One example of the Latent Diffusion Model is StableDiffusion (the one that lets you generate images from the text).**



# Latent Diffusion Model Architecture



# Code Demo

## UNet Diffusion Model on Fashion MNIST Dataset:

[https://github.com/rahulbhadani/CPE487587\\_SP26/blob/master/Code/Chapter\\_08\\_DiffusionModel.ipynb](https://github.com/rahulbhadani/CPE487587_SP26/blob/master/Code/Chapter_08_DiffusionModel.ipynb)

**UNet Diffusion Model on Digit MNIST Dataset:** <https://colab.research.google.com/drive/1RqgzkJEL-W3lUOhUkzAoEcVlnM0Cu1Fp?usp=sharing>

# ODE, Flow, and Dynamical Systems

# Ordinary Differential Equations

A first order ODE:

$$y'(t) = \frac{dy}{dt} = f(t, y(t))$$

with initial condition  $y(0) = y_0$ .

In general, an ODE is a functional relation of the form

$$F(t, x, x^{(1)}, \dots, x^{(k)}) = 0$$

where  $x^{(j)}(t) = \frac{d^j x(t)}{dt^j}$  the j-th derivative.

$t$  is called the independent variable, and  $x$  is the dependent variable. The highest derivative is the order of the ODE.

# A System of ODE

$$x_1^{(k)} = f_1(t, x, x^{(1)}, \dots, x^{(k-1)})$$

$$x_2^{(k)} = f_2(t, x, x^{(1)}, \dots, x^{(k-1)})$$

$$\vdots$$

$$x_n^{(k)} = f_n(t, x, x^{(1)}, \dots, x^{(k-1)})$$

# A linear system of ODE

A system of ODE is linear if it is of the form

$$x_i^{(k)} = g_i(t) + \sum_{l=1}^n \sum_{j=0}^{k-1} f_{i,j,l}(t) x_l^{(j)}$$

Further, it is **homogenous** if  $g_i(t) = 0$ .

# First-order ODE System

Any system can always be reduced to a first-order system by changing to the new set of dependent variables

$\gamma = (x, x^{(1)}, \dots, x^{(k-1)})$  which yields

$$\dot{\gamma}_1 = \gamma_2$$

$$\dot{\gamma}_2 = \gamma_3$$

$$\vdots$$

$$\dot{\gamma}_{k-1} = \gamma_k$$

$$\dot{\gamma}_k = f(t, \gamma)$$

# First-order ODE System

Any system can always be reduced to a first-order system by changing to the new set of dependent variables

$y = (x, x^{(1)}, \dots, x^{(k-1)})$  which yields

$$\dot{y}_1 = y_2$$

$$\dot{y}_2 = y_3$$

$$\vdots$$

$$\dot{y}_{k-1} = y_k$$

$$\dot{y}_k = f(t, y)$$

We can even add  $t$  to the dependent variables  $z = (t, y)$ , making the right hand side independent of  $t$ . Hence

$$\dot{z}_1 = 1$$

$$\dot{z}_2 = z_3$$

$$\vdots$$

$$\dot{z}_k = z_{k+1}$$

$$\dot{z}_{k+1} = f(z)$$

Such a system where  $f$  doesn't depend on  $t$  is called **autonomous** system.



# First-order Autonomous Equations

The solution starting at a certain point  $x_0$  at time  $t = 0$  is

$$\dot{x} = f(x), \quad x(0) = x_0$$

It is also called **Initial Value Problem**.

We can have a solution  $\phi(t)$  with  $\phi(0) = x_0$  and the solution  $\psi(t)$  with  $\psi(t_0) = x_0$  for a general time  $t_0$  is given by a simple shift  $\psi(t) = \phi(t - t_0)$ . This holds for any autonomous equation.

# Dynamical Systems

Think of a dynamical system as the time evolution of some physical system, such as the motion of a few planets under the influence of their respective gravitational forces

Usually we want to know the fate of such a system for long times, like will planet eventually collide or will the system persist for all times?

Such analysis is embedded in **dynamical systems theory**. A dynamical system is a map

$$T : G \times M \rightarrow M$$

$M$  is a set and  $G = \mathbb{N}_0$  or  $G = \mathbb{Z}$  for discrete dynamical system and  $G = \mathbb{R}$  for a continuous dynamical system.

# References

- ⚡ **Original Paper** on Diffusion Model, Deep Unsupervised Learning using Nonequilibrium Thermodynamics: <https://proceedings.mlr.press/v37/sohl-dickstein15.pdf>
- ⚡ **The Most Influential Paper:** Denoising Diffusion Probabilistic Models  
<https://arxiv.org/pdf/2006.11239>
- ⚡ Deep Learning Notes by Prof. LI Hongsheng: <https://dl.ee.cuhk.edu.hk/slides/diffusion.pdf>
- ⚡ Diffusion Models: A Comprehensive Survey of Methods and Applications:  
[https://dl.acm.org/doi/full/10.1145/3626235?casa\\_token=tLriJSOLl7EAAAAA:G\\_ue\\_8JwODm\\_PGRul\\_bBdK70QpTLetomFM1ioVMzebrqCaAtiozqPpcs2hV62SMAUIz01-TvweBV](https://dl.acm.org/doi/full/10.1145/3626235?casa_token=tLriJSOLl7EAAAAA:G_ue_8JwODm_PGRul_bBdK70QpTLetomFM1ioVMzebrqCaAtiozqPpcs2hV62SMAUIz01-TvweBV)
- ⚡ Understanding Diffusion Models: A Unified Perspective: <https://arxiv.org/pdf/2208.11970.pdf>
- ⚡ U-Net: Convolutional Networks for Biomedical Image Segmentation:  
<https://arxiv.org/pdf/1505.04597.pdf>
- ⚡ ConvNext: A ConvNet for the 2020s: <https://arxiv.org/pdf/2201.03545.pdf>
- ⚡ ACC-UNet: A Completely Convolutional UNet model for the 2020s:  
<https://arxiv.org/pdf/2308.13680.pdf>
- ⚡ Latent Diffusion Paper: <https://arxiv.org/pdf/2112.10752.pdf>

# References

- ⚡ Stable Diffusion Github Repo: <https://github.com/CompVis/stable-diffusion?tab=readme-ov-file>
- ⚡ Diffusion Models Beat GANs on Image Synthesis (Guided Diffusion): <https://arxiv.org/pdf/2105.05233.pdf>
- ⚡ Guided Diffusion Github Repo: <https://github.com/openai/guided-diffusion>

# The End